Reg No: 241501241

Experiment – 14 : Pro-Code Extension using Zoho Catalyst

Aim

To study how pro-code development using Zoho Catalyst can extend a low-code application such as Event Desk by supporting custom backend logic, advanced processing, API-based integration, and additional functionality beyond the capabilities of the existing Zoho Creator configuration..

Theory

Pro-code development involves writing application logic using programming languages, frameworks, and development tools to implement functionality that may not be conveniently achievable through visual development alone.

Zoho Catalyst is a cloud-based development platform that provides tools for building and deploying custom applications and backend services. It can complement Zoho Creator by allowing developers to implement specialized logic and expose functionality through APIs.

In the Event Desk platform, Zoho Creator handles participant registration, reports, dashboards, and workflows. If additional processing or integration requirements arise, a separately developed Catalyst service may be considered.

The following concepts are relevant to the platform:

1. Low-Code and Pro-Code Integration

Zoho Creator can provide the main registration interface, while a Catalyst application can provide additional backend functionality. The two platforms can communicate through appropriately configured APIs.

2. Custom Backend Logic

Pro-code services can implement complex validation, specialized calculations, data transformation, or processing logic that requires greater programming flexibility.

3. API-Based Communication

APIs allow applications to exchange requests and responses. A Catalyst service could receive a request from Event Desk, process the required information, and return a structured response.

4. Authentication and Authorization

Communication between Event Desk and Catalyst should use appropriate authentication and authorization mechanisms. Credentials must be stored securely, and access should be limited to permitted operations.

5. Data Processing and Validation

A custom service can validate incoming data, check required fields, apply business rules, and prepare results before returning them to the calling application.

6. Error Handling and Logging

Pro-code services can include structured error handling and logging to help identify invalid requests, processing failures, and integration issues.

7. Scalability and Modularity

Separating specialized processing into a dedicated service can make the application architecture more modular. Actual scalability depends on the selected Catalyst services, configuration, workload, and platform limits.

8. Event Desk Integration Possibilities

Potential extensions include advanced registration validation, additional analytics processing, automated reporting, or integration with external services. These are possible extensions and should not be considered existing features unless they have been implemented.

9. Testing and Deployment

A Catalyst service should be developed and tested separately before connecting it to the live Event Desk application. API requests, response formats, permissions, and error conditions should be verified before deployment.

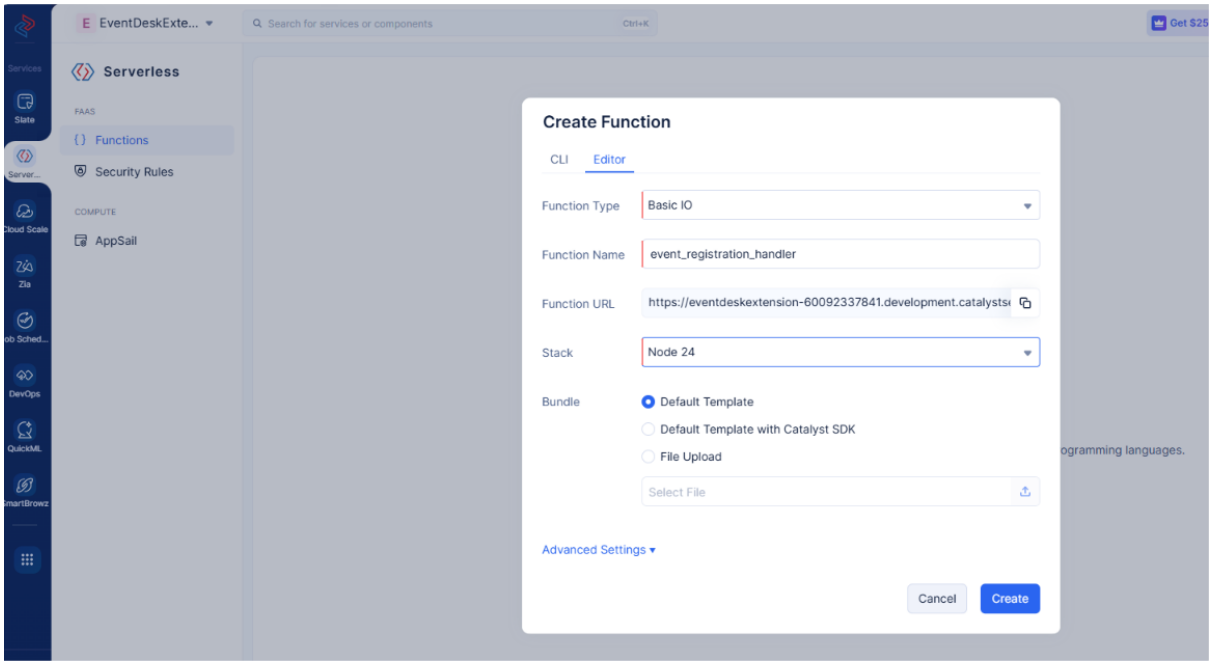
Together, Zoho Creator and Zoho Catalyst can support a hybrid development approach in which low-code tools handle standard application functions and pro-code services provide specialized capabilities.

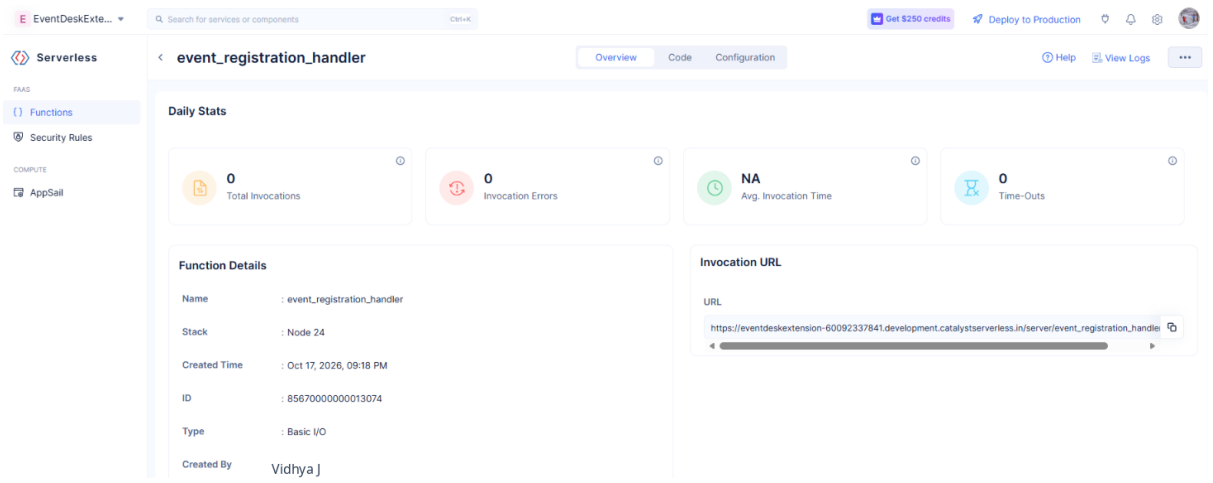
Procedure

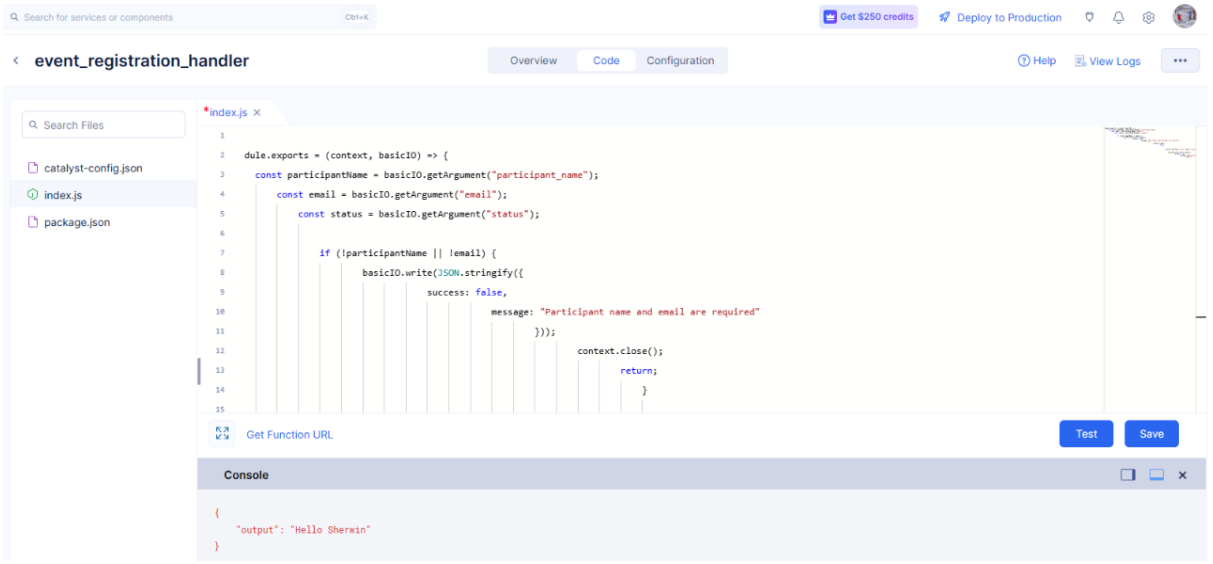
1. Open the Event Desk application in Zoho Creator and review its existing forms, reports, dashboards, and workflows.

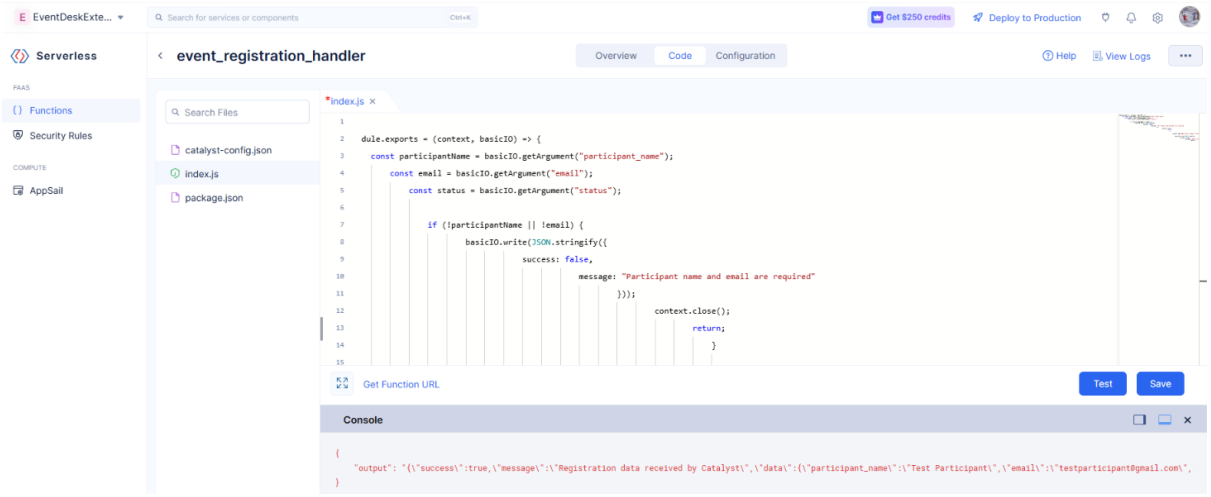
2. Identify a requirement that may benefit from custom backend processing beyond the current low-code configuration.
3. Review the available Zoho Catalyst services and select an appropriate application or function type for the proposed extension.
4. Access the Zoho Catalyst development environment and review the project creation options.
5. Create a separate development project if implementing a practical prototype.
6. Select a supported programming language or runtime and design the required backend function.
7. Define the API endpoint, request parameters, expected response format, and validation rules.
8. Implement the required business logic using the selected programming environment.
9. Add error handling for invalid input, failed processing, and unexpected requests.
10. Configure authentication and authorization for the service using supported security mechanisms.
11. Test the service independently using valid and invalid sample requests.
12. Review the API response to confirm that the output matches the expected format.
13. If integrating with Event Desk, configure the required API call or connection using the supported Zoho Creator integration mechanism.
14. Test the integration in a development or test environment before using actual participant records.
15. Document the service's purpose, endpoint requirements, security configuration, and deployment steps.

Screenshots


Serverless
FAAS
Functions
Security Rules
COMPUTE
AppSail
Create Function
CLI
Editor
Function Type
Basic IO
Function Name
event_registration_handler
Function URL
https://eventdeskextension-60092337841.development.catalyst
Stack
Node 24
Bundle
Default Template
Default Template with Catalyst SDK
File Upload
Select File
Advanced Settings
Cancel
Create


event_registration_handler
Overview
Code
Configuration
Daily Stats
0
Total Invocations
0
Invocation Errors
NA
Avg. Invocation Time
0
Time-Outs
Function Details
Name : event_registration_handler
Stack : Node 24
Created Time : Oct 17, 2026, 09:18 PM
ID : 85670000000013074
Type : Basic I/O
Created By Vidhya J
Invocation URL
URL


event_registration_handler
Overview
Code
Configuration
Search Files
catalyst-config.json
index.js
package.json
*index.js
dule.exports = (context, basicIO) => {
const participantName = basicIO.getArgument("participant_name");
const email = basicIO.getArgument("email");
const status = basicIO.getArgument("status");
if (!participantName || !email) {
basicIO.write(JSON.stringify({
success: false,
message: "Participant name and email are required"
}));
context.close();
return;
}
Get Function URL
Test
Save
Console
{
"output": "Hello Sherwin"
}



Result

The role of Zoho Catalyst in extending a low-code application through pro-code development was studied in the context of Event Desk. The approach can support custom backend logic, advanced validation, API-based communication, and specialized processing beyond the existing visual configuration. A hybrid architecture can combine Zoho Creator's forms, reports, dashboards, and workflows with separately developed services. Successful integration depends on implementing and testing the Catalyst service, configuring secure communication, and validating its behavior with Event Desk.